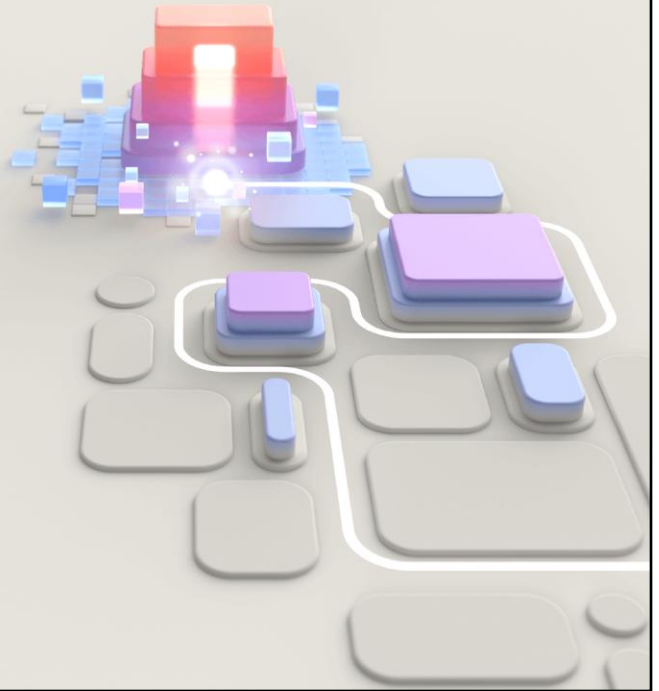




PL-400T00

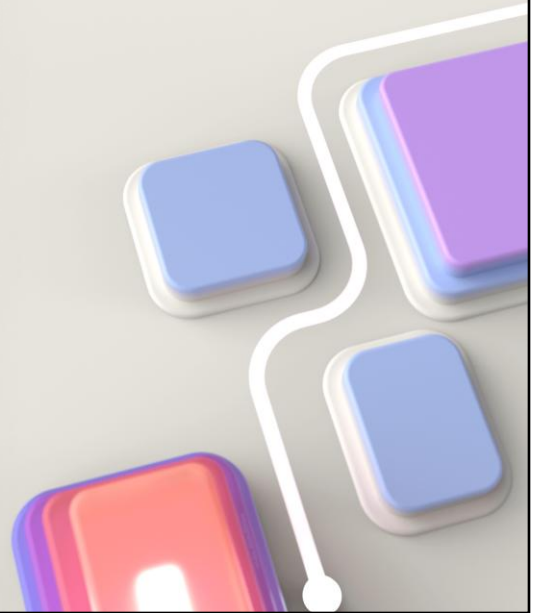
Microsoft Power Platform Developer

© Copyright Microsoft Corporation. All rights reserved.



Application lifecycle management (ALM)

© Copyright Microsoft Corporation. All rights reserved.



Implement application lifecycle management (ALM)

- Create solutions and manage solution components
- Import and export solutions
- Manage solution dependencies
- Create and use environment variables
- Describe the difference between unmanaged and managed solutions
- Manage solution layers

Modules



- Solutions and Application Lifecycle Management

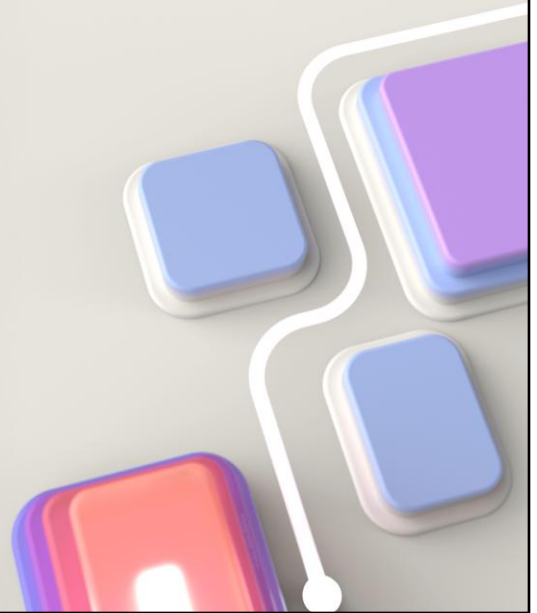
© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Solutions and Application Lifecycle Management

Module 1: Solutions and Application Lifecycle Management

© Copyright Microsoft Corporation. All rights reserved.



Application Lifecycle Management

Learn the importance of ALM in the Power Platform

In this module we will

- Add and remove apps, flows, and tables in a solution
- Examine solution dependencies
- Introduce environment variables
- Examine solution and app checkers
- Import and Export solutions
- Describe how to automate solution management

Learning objectives

- Package existing items into a solution.
- Import and export solutions.
- Learn about component dependency on other components

Learning Objectives

After completing this module, you will be able to:

- 1 Manage solutions in Power Apps and Power Automate
- 2 Discover the concept of solution layering and version control

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn modules

Manage solutions in Power Apps and Power Automate

<https://learn.microsoft.com/training/modules/manage-solutions-power-automate>

Introduction to solutions for Microsoft Power Platform

<https://learn.microsoft.com/training/modules/introduction-solutions>



© Copyright Microsoft Corporation. All rights reserved.

Link to student courseware on Learn

Manage solutions in Power Apps and Power Automate

<https://learn.microsoft.com/training/modules/manage-solutions-power-automate>

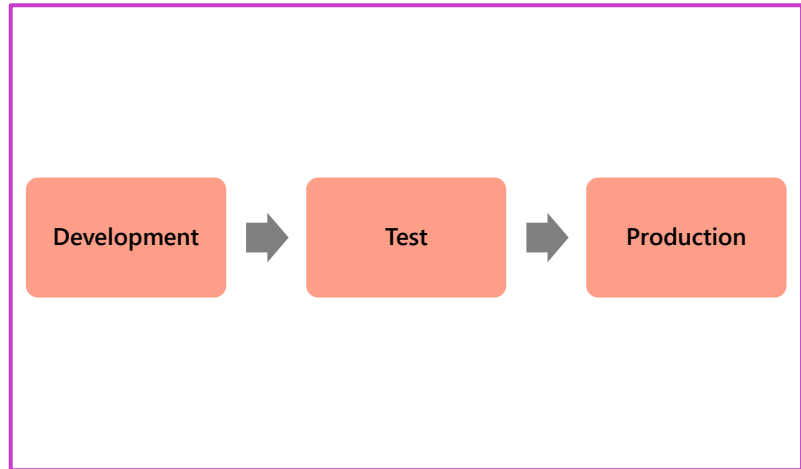
Introduction to solutions for Microsoft Power Platform

<https://learn.microsoft.com/training/modules/introduction-solutions>

Instructor: The ILT content has been cut down from the modules to focus on the PL-400 skills measured. Remember solutions were introduced in Learning Path 1

Solutions

- Solutions are containers of Power Platform components
- Transport customizations from one environment to another
- Part of an overall strategy for application lifecycle management (ALM)



© Copyright Microsoft Corporation. All rights reserved.

Remember this from Dataverse learning path.

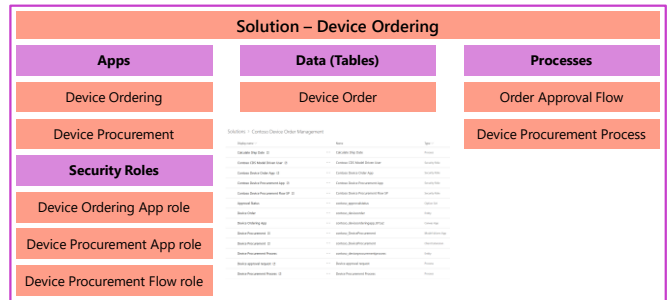
In that learning path we just created an (unmanaged) solution to hold all our tables, apps, and flows.

We will now look deep at solutions

Solutions allow you to package features, such as apps in Microsoft Power Apps (canvas and model-driven), site maps, flows, tables, forms, custom connectors, web resources, choices, charts, and columns, to transport from one environment to another. Only the metadata, such as tables and their columns and configuration data, are transported. No business data is transported.

Solutions are part of the overall application lifecycle management (ALM) practice of Microsoft Power Platform. Solutions are important mechanisms for implementing ALM.

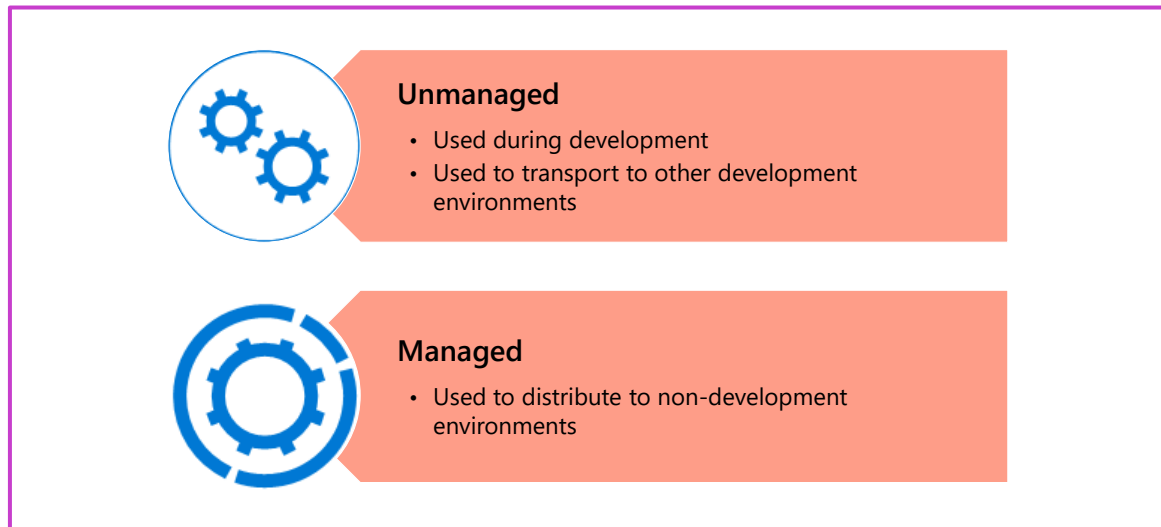
Example solution



© Copyright Microsoft Corporation. All rights reserved.

This is app in a day scenario

Types of solution



© Copyright Microsoft Corporation. All rights reserved.

Solutions are mechanisms for implementing ALM in Power Apps and Power Automate. A solution can be managed or unmanaged.

Unmanaged solutions are used while you're still in your development environment in the initial design, build, and test phases. Unmanaged solutions should be considered your source for Microsoft Power Platform assets. When an unmanaged solution is deleted, only the solution container of any included customizations is deleted. All unmanaged customizations will remain in effect and belong to the default solution.

Managed solutions are used to deploy to any environment that isn't a development environment for that solution. These environments include test, user acceptance testing (UAT), system integration testing (SIT), and production. To make it easier and faster, managed solutions can be created independently from other managed solutions in the same environment. As an ALM best practice, managed solutions should be generated by exporting an unmanaged solution as managed and then considered a build artifact.

Changes can only be made to unmanaged solutions. If you need to modify a managed solution, you need to add them to an unmanaged solution. As a result, you will create a dependency between your unmanaged customizations and the managed solution. When a dependency exists, the managed solution can't be uninstalled until you remove the dependency. Some managed components can't be edited. To verify whether a component can be edited, view the **Managed** properties.

You can't export a managed solution. Also, when a managed solution is deleted (uninstalled), all included customizations and extensions are removed.

Default solutions

- Common Data Service Default solution
- Default solution
- From the maker portal you can now set your preferred solution

© Copyright Microsoft Corporation. All rights reserved.

Common Data Service Default Solution. This solution is available for makers to use by default for their customizations in an environment. The Common Data Service Default Solution is useful when you want to evaluate or learn Power Apps. However, we recommend that app makers work in their own unmanaged solutions.

Default Solution. This is a special solution that contains all components in the system. The default solution is useful for discovering all the components and configurations in your system.

However, we recommend that you create a solution to manage your customizations.

You can now also set your preferred solution :
<https://learn.microsoft.com/en-us/power-apps/maker/data-platform/preferred-solution>

Solution components

- 1** Solution components are categorized to track the individual items associated with the solution

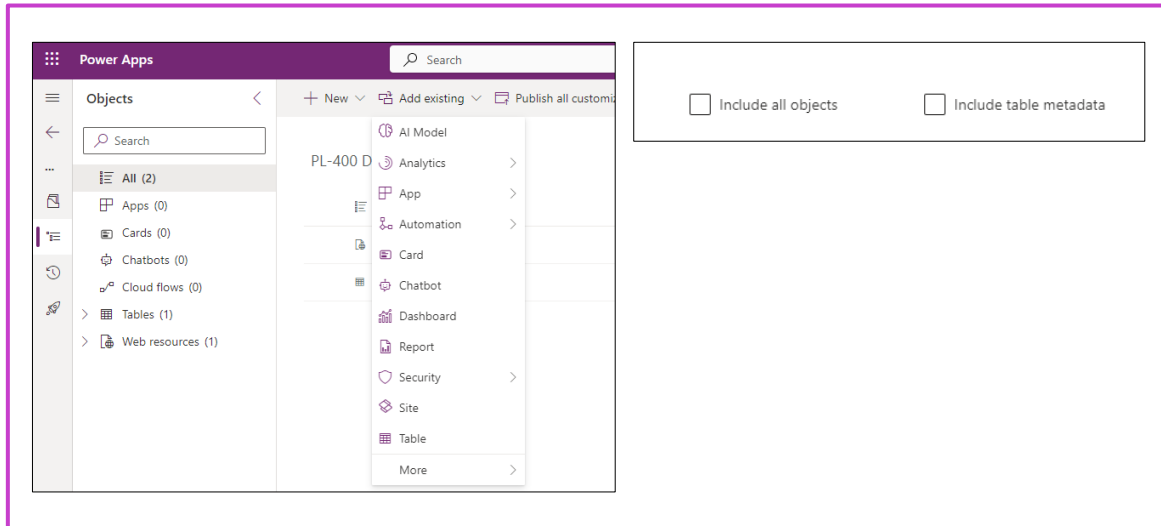
- 2** Component types include tables, apps (model and canvas), Power Automate flows, security components as well as developer assets

- 3** Some components are sub-components, for example forms, views, charts, and columns are all assets of tables

- 4** Reference data is not included as a solution component

© Copyright Microsoft Corporation. All rights reserved.

Segmented solutions

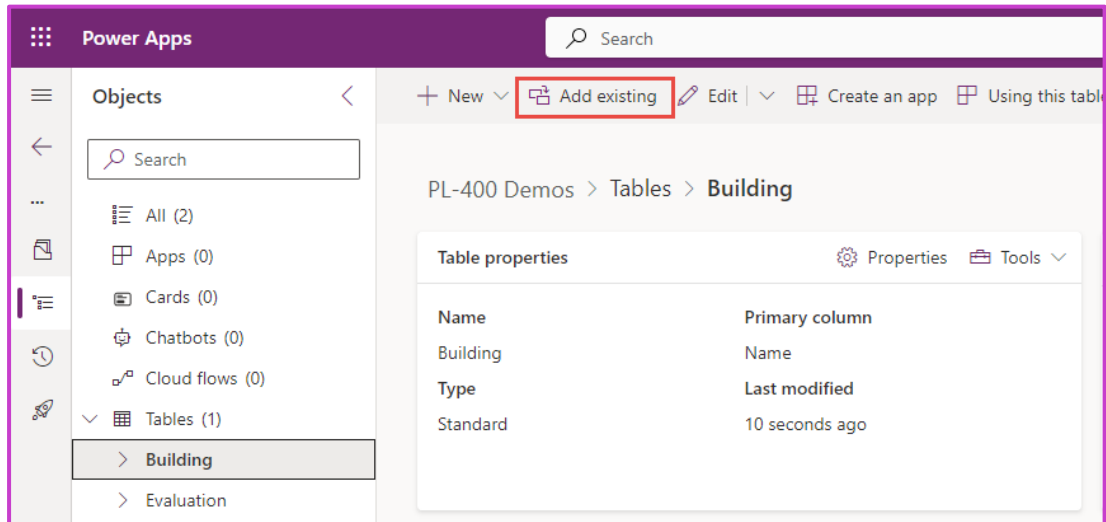


© Copyright Microsoft Corporation. All rights reserved.

You generally do not want to check the Include all table components/assets.

The primary scenario for using include all objects is if a new table is created in the Dataverse environment but wasn't originally added to this solution and you now want to include it. If you are editing an existing table, it is not recommended to include all objects, and just add the assets that you are changing.

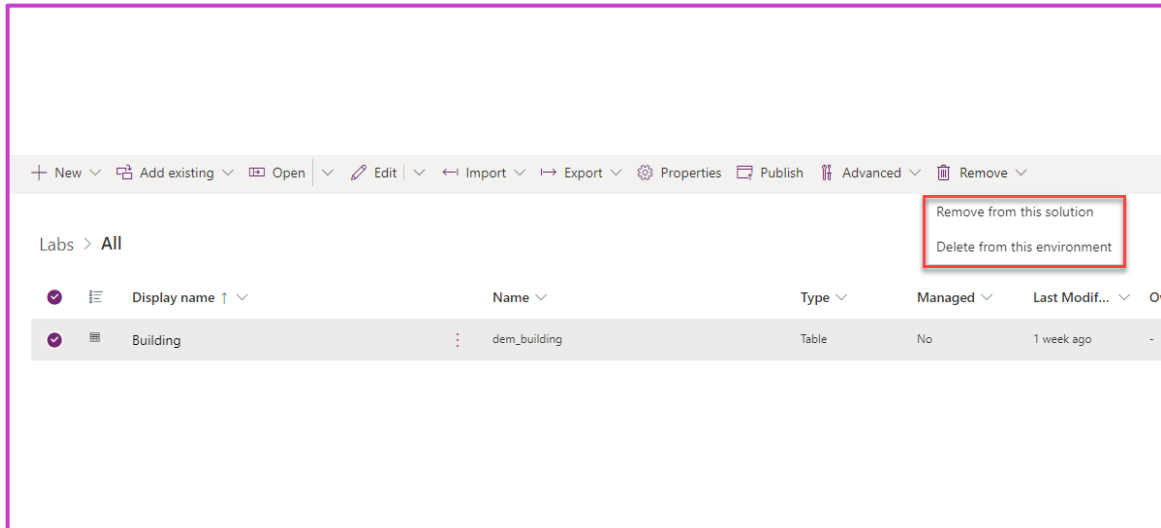
Add table assets



© Copyright Microsoft Corporation. All rights reserved.

To add existing entity solution components (like columns, forms and views) edit the specific table and choose "Add existing".

Removing Items



© Copyright Microsoft Corporation. All rights reserved.

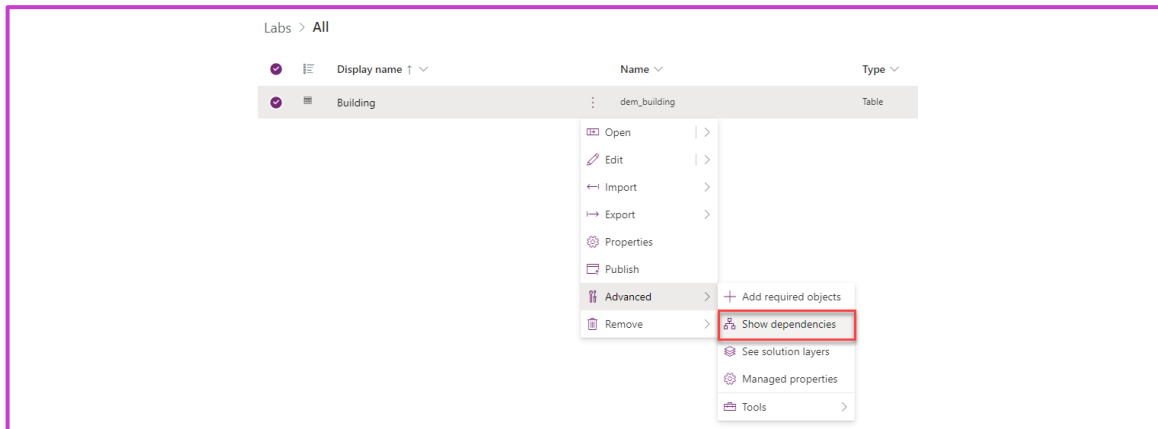
There are two options for removing components from a solution; Remove and Delete.

- Remove simply removes the reference from the solution to the component, however the component still exists in the Dataverse environment. A component that is removed can be re-added using Add Existing.
- Delete on the other hand, physically removes the component from the Dataverse environment and all other solutions referencing that component.

Solution dependencies

Solution components can depend on other objects

Solution components can be required by other objects



© Copyright Microsoft Corporation. All rights reserved.

Solutions are made up of a set of components. These components often depend on other components to work. For example, if you create a dashboard that has a list of Account records on it, that dashboard component is dependent on that table being present when the solution is installed, or it must be included in the same solution as the dashboard.

Component dependencies can be resolved by an item in the same solution, or they can be resolved by an external solution. When resolved by an external solution, that creates a solution to solution dependency.

Environment variables

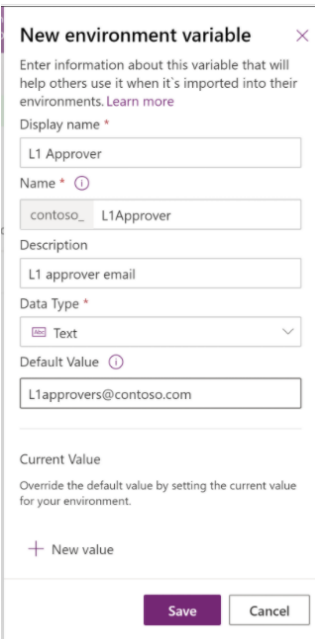
- 1** Apps and flows often require different configuration settings across environments e.g., Sender email, Portal URL etc.
- 2** Flows become configurable without hard coded values that must be edited upon deployment
- 3** Supports default values that can be overridden in each environment
- 4** Environment variables are solution components and participate in full application lifecycle management

© Copyright Microsoft Corporation. All rights reserved.

Creating environment variables

- Decimal number, Text, JSON, Yes/No or a secret
- Current value overrides any default value provided
- Remove the current value if you don't want it transported to the next level environment

© Copyright Microsoft Corporation. All rights reserved.



The screenshot shows a 'New environment variable' dialog box with a close button (X) in the top right corner. The dialog contains the following fields and sections:

- Display name ***: A text input field containing 'L1 Approver'.
- Name ***: A text input field containing 'contoso_ L1Approver'.
- Description**: A text input field containing 'L1 approver email'.
- Data Type ***: A dropdown menu with 'Text' selected.
- Default Value**: A text input field containing 'L1approvers@contoso.com'.
- Current Value**: A section with the text 'Override the default value by setting the current value for your environment.' and a '+ New value' link.
- Buttons**: 'Save' and 'Cancel' buttons at the bottom right.

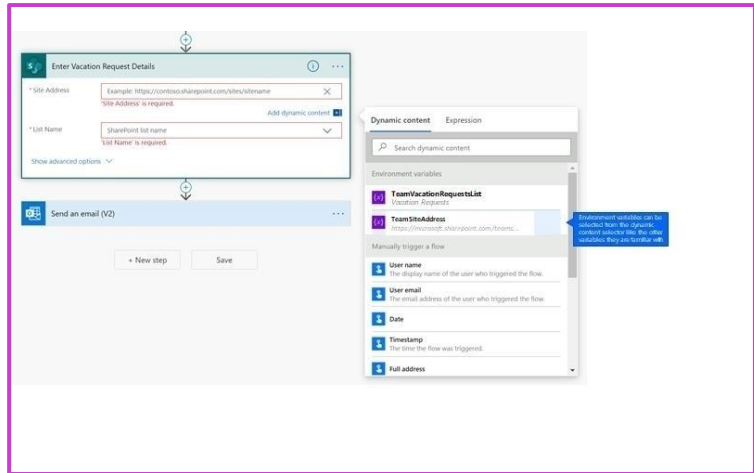
Environment variables can have different values when re-used, enter information about this variable so that future users can understand its purpose

By default, the maker portal import process will always ask you to check / specify a value

Using setting files, from an automation standpoint - developers frequently need to do this:
<https://learn.microsoft.com/en-us/power-platform/alm/conn-ref-env-variables-build-tools>

Using environment variables

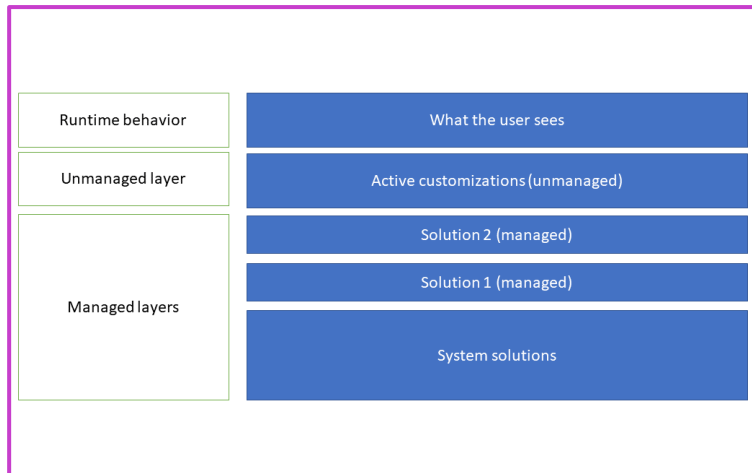
- Use the dynamic content panel to select variable values
- No need to manually query
- Use environment variable values in trigger properties
- Does not require a Microsoft Dataverse license to use variables



© Copyright Microsoft Corporation. All rights reserved.

Solution layers

- Managed layer(s)
- Unmanaged (Active) layer
- Editing or even just testing a managed flow without saving creates an unmanaged layer
- Delete active layer



© Copyright Microsoft Corporation. All rights reserved.

Managed layers

Managed layers. All imported managed solutions and the system solution exist at this level.

- The system layer is at the base of the managed layers level. The system layer contains the entities and components that are required for the platform to function.
- All imported, managed solutions and the system solution exist at the managed level.
- When multiple managed solutions are installed, the last one that's installed is above the managed solution that was previously installed. Essentially, the second solution that's installed can customize the one that was installed before it.
- When two managed solutions have conflicting definitions, the runtime behavior is "last one wins" or a merge logic is implemented.
- If you uninstall a managed solution, the managed solution below it will take effect.
- If you uninstall all managed solutions, the default behavior that's defined within the system solution will be applied.

Unmanaged layer

Unmanaged layer. All imported unmanaged solutions and unmanaged customizations exist at this layer. The unmanaged layer is a single layer.

All imported unmanaged solutions and impromptu customizations exist at the unmanaged layer. All unmanaged solutions share a single, unmanaged layer.

Certain components are available that have a merge behavior, such as site maps and forms. Each layer is merged to produce the runtime behavior as opposed to the "last layer wins" behavior.

Solution checker

- 1 As Power Automate and Power Apps are used to customize a deployment, each offers their own inline app checkers that are helpful for real-time issue resolution.
- 2 Solution checker is able to look at the whole solution, do static analysis, and produce a detailed list of any issues found.
- 3 Solution checker should be run regularly on any unmanaged solution you are building in your development environments.

© Copyright Microsoft Corporation. All rights reserved.

As Power Automate and Power Apps are used to customize a deployment, each offers their own inline app checkers that are helpful for real-time issue resolution. However, solution checker (more details [here](#)) is able to look at the whole solution, do static analysis, and produce a detailed list of any issues found.

Solution checker should be run regularly on any unmanaged solution you are building in your development environments. Solution checker can analyze Power Apps and Power Automate flows as well as code assets like plugins that developers create. The project team can manually run solution checker from the maker portal by selecting the solution and then running the checker. Solution checker can also be automated to run as part of a build process either using PowerShell or Azure Pipeline tasks. By automating the run of solution checker, it can become an integral part of the build process and even can be set up to stop the build if too many errors have occurred. Simply running solution checker is not enough for the team to be successful they must also have a plan in place to regularly evaluate and resolve problems identified.

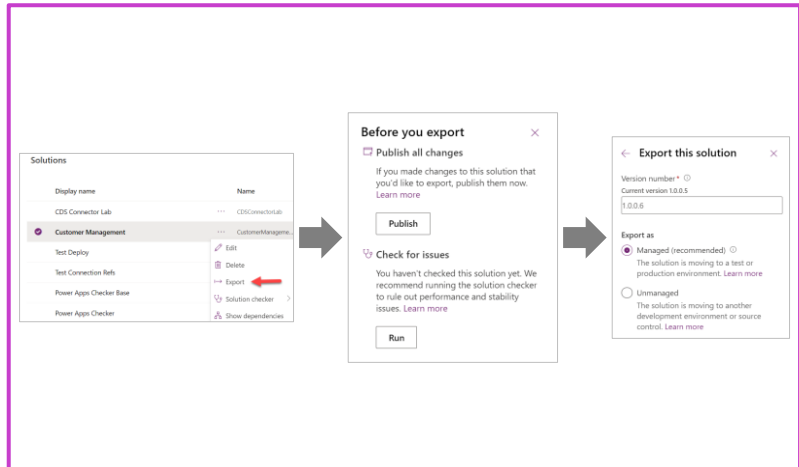
Manual export

Always export both managed and unmanaged

Managed solutions are deployed read-only to test/production

Unmanaged solutions can be used to restore your dev environment or store in source control

Publish all and check for issues should be part of your export process



© Copyright Microsoft Corporation. All rights reserved.

Manual import

Allows hooking up each connection reference to a valid connection or setting up new connections as required

Allows setting any environment variables

Connections and environment variable values are retained during updates

© Copyright Microsoft Corporation. All rights reserved.

The image shows a sequence of three screenshots of the 'Import a solution' dialog box, connected by downward arrows. The first screenshot shows the 'Select a file' step with a 'Browse' button. The second screenshot shows the 'Connections' step with a list of connections and a 'Refresh' button. The third screenshot shows the 'Environment Variables' step with input fields for 'CanvasURL' and 'ModelURL'.

Import a solution

Select a file
Drag and drop or browse for the solution file to import.

Browse No file chosen

Import a solution

Refresh

Connections
Re-establish connections to activate your solution. If you create a connection you must Refresh. You will not lose your report programs.

2 updates needed

Common Data Service (current environ...
[Link] [Refresh]

Office 365 Outlook +
[Link] [Refresh]

Import a solution

Environment Variables
Enter information for each field, so your app works properly. You can edit your environment variables later.

2 updates needed

CanvasURL
Enter text
Please enter a value

ModelURL
Enter text
Please enter a value

When you do a manual import if there are any connections in use you'll be prompted to establish the connection references

if the solution has any environment variables you will be given a chance to provide values these values will be stored as the current value for that definition

Importing a new version of a managed solution

- 1 **Upgrade** – Default option, imports changes and applies them immediately including removing components not in new solution, old solution and any patches are removed. Recommended.

- 2 **Stage for Upgrade** – Similar to upgrade but it pauses after new solution is imported before removing components so you can do data migration etc. and then manually apply the solution.

- 3 **Update** – Applies changes, no removal of items occur. Not recommended.

© Copyright Microsoft Corporation. All rights reserved.

<https://docs.microsoft.com/en-us/power-apps/maker/data-platform/update-solutions>

Non-solution/environment aware components

- Developers will often work with assets that cannot be bundled in solutions
- Examples:
 - **Configuration data** – Use the [Configuration Migration Tool](#)
 - **Power BI reports / visualizations** – Consider [Power BI Deployment pipelines](#)
 - **Microsoft Azure resources** – Use Bicep or Azure RM templates
 - **External services** – Consult vendor documentation
- Some component types may also require manual deployment
 - Service endpoints
 - Web hooks
- Azure DevOps can help to “unify” complex deployments

© Copyright Microsoft Corporation. All rights reserved.

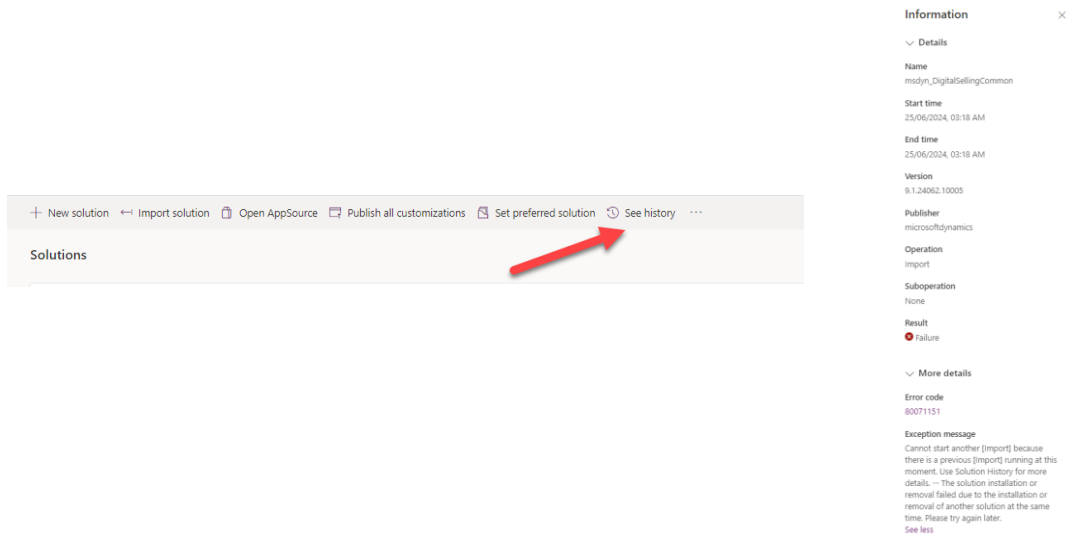
Most components that developers work with can be bundled into a solution and deployed straightforwardly. But there will be occasions where additional tooling or thought is required, especially if the deployment is being carried out manually:

- **Configuration data:** Basically, any table data that is used or required as part of the app or solution. This is especially a concern if automations are being used, such as Business Rules, flows or workflows, as these may not switch on correctly if the expected record / GUID is not present in the target environment. By running a CMT export / import, data can be moved, and records can be persisted with the same ID's. This very often comes up as a requirement when working with Dynamics 365 applications, such as Field Service or Project Operations
- **Power BI reports / visualizations:** Although it's possible to add the reference to these components into a solution, these components will always exist separately in a workspace. It will be necessary to create an environment variable that references the workspace location for each environment, but the process for deploying out the Power BI content will need to be carried out manually or automated using deployment pipelines (premium capacity workspaces are required for this).
- **Microsoft Azure resources:** Azure has it's own separate ALM processes that needs to be considered carefully. Generally speaking, ensuring you can mirror the environment structure in Azure via resource groups is recommended. Assets can always be deployed manually, but it's recommended that developers leverage either Bicep or Azure RM templates and store this within a repository.
- **External services:** Most Power Platform solutions will integrate with one or several third-party tools, and there may be separate processes that need to be followed for any changes. It's impossible to cover all of these off, so developers should always consult with relevant documentation or teams within the organisation. In simple terms, good communication is the name of the game here.

There are also some specific component types that may not “fit” nicely, especially if environment components are very clearly separated. For example, if you are working with service endpoints across three environments (Development, UAT and Production). It's not possible to easily change a service endpoint configuration in PRT to a different URL when deploying out. You may need to write some custom scripting to replace this during a deployment or consider instead to configure these outside the context of a solution.

Azure DevOps is a great tool for allowing you to bring together all these considerations into a single, discrete pipeline for deployments. For example, you could have a pipeline that deploys out your Dataverse solutions and then all associated Azure resources.

Troubleshooting solution deployment errors



© Copyright Microsoft Corporation. All rights reserved.

The maker interface will always notify you in case a solution deployment error occurs, with some detail provided via a log. This is also stored and is made available in the Solution history view (see example screenshots above).

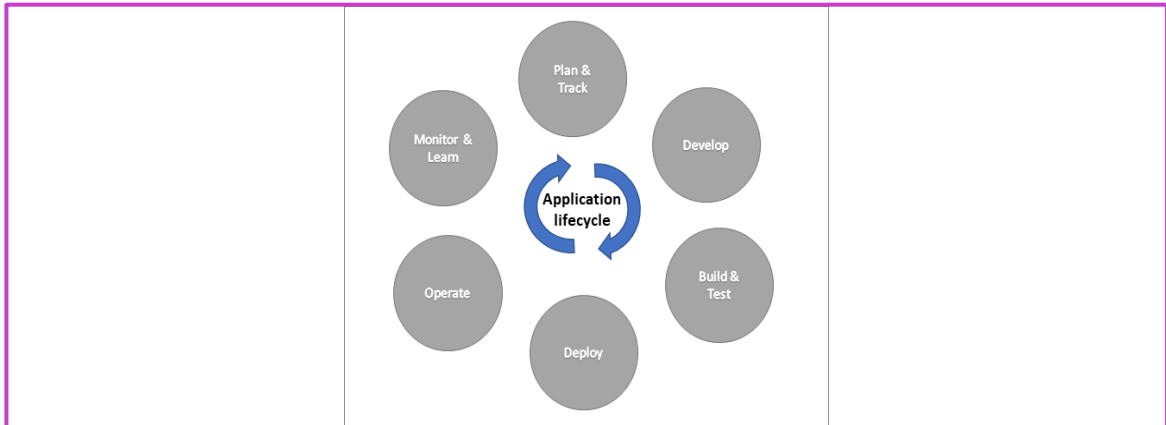
Common reasons for deployment errors include:

- **Missing components:** If a required component is missing either a) from the solution or b) from the environment itself, then the import will fail. The log will typically notify you as to what components are missing, so you can take corrective action.
- **Another solution import:** Dataverse only supports one active solution import job at any time. If another developer or Microsoft are deploying at the same time, you will likely get the same or similar error as indicated in the screenshot above.
- **Holding solution already exists:** If a previous upgrade has not been applied, then attempting to import another version of the same holding solution will result in an error.
- **Component is required:** When performing an uninstall, if a component is referenced (i.e. has a dependency), then an error will occur. You will need to inspect and remove the dependency, before re-attempting the deployment.

This list is not exhaustive. In most cases, the error message will provide a clear indication of what needs to be done to resolve. In worst case scenarios, you may need to open a ticket with Microsoft support.

Automate solution management

So far in this learning path you have seen how solutions can be used to track your app components and transport them from one environment to another manually.



© Copyright Microsoft Corporation. All rights reserved.

So far in this learning path you have seen how solutions can be used to track your app components and transport them from one environment to another manually. In this topic, we will look at how automating some of these actions can help implement a more robust application lifecycle management (ALM) for your project. ALM involves more than just the building of the app, it includes the governance (for example, requirements management), the development, and the maintenance. Solutions play a key role in an ALM strategy along with tools like Azure DevOps or GitHub.

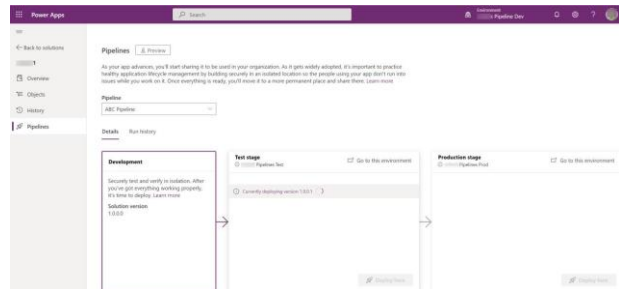
On projects, the solution architect typically would outline the ALM strategy, and a DevOps focused resource would implement the automation. As a team member you would participate in the ALM process by managing your work items in the chosen tool and using solutions to track your app and flow changes. The solution would then be put into the ALM automated processes that would start a build and release process to move your changes automatically from dev to test to production, with appropriate approvals along the way. Think of the application lifecycle as a cyclical app development process that involves these areas: plan and track, develop, build and test, deploy, operate, monitor, and learn from discovery.

[!div class="mx-imgBorder"] [Diagram of the application lifecycle development process.](#)

While all the actions can be done manually, automation ensures they are done consistently each time they're performed. They also can still happen if, for example, the person who performs the manual actions is out sick. DevOps focused tools like Azure DevOps or GitHub provide the automation, but also provide work item tracking and source control.

Automating ALM with Power Platform Pipelines

- Ideal starting off point for non-complex solution deployments
- Key advantages:
 - Easy to configure and get started
 - Supports extensible approval / gating processes
 - Supports redeployment of previous solution versions
- Two types:
 - **Personal:** For developer environments
 - **Standard:** Requires managed environments (premium licensing)



© Copyright Microsoft Corporation. All rights reserved.

Pipelines in Power Platform aim to democratize application lifecycle management (ALM) for Power Platform and Dynamics 365 customers by bringing ALM automation and continuous integration and continuous delivery (CI/CD) capabilities into the service in a manner that's more approachable for all makers, admins, and developers.

- **Easy to configure and get started:** Using the built-in model-driven app, both makers and developers can straightforwardly configure a deployment process to one or multiple environments, covering all required solution(s)
- **Supports extensible approval / gating processes:** Via Power Automate integration, it's possible to build out more complex approval steps before each deployment occurs. For example, a modern approval could be sent through to a release manager in the organisation via Teams.
- **Supports redeployment of previous solution versions:** For situations where an update breaks a working solution, this can be easily reverted by deploying a previous version of the solution.

Developers should be aware of the two different types of pipelines available:

- **Personal:** As the name implies, this allows for a pipeline to be configured to deploy to up to two target environments. The key requirement is that the source environment must be a developer environment
- **Standard:** This has no restrictions on the number of target environments or on the underlying source environment type, but all target environments must be enabled as managed environments. Managed Environments are a suite of premium capabilities that allows admins to manage Power Platform at scale with more control, less effort, and more insights. They require additional premium licensing for all users accessing the environment.

Consider doing a demonstration if time allows, using Personal pipelines:

<https://learn.microsoft.com/en-us/power-platform/alm/set-up-pipelines>

Automating ALM with Source Control

- Source control is a feature of Azure DevOps and GitHub that allows you to keep track of versions of your solution and code components.
- Automation is important because it brings consistency to what would otherwise be a manual process.

© Copyright Microsoft Corporation. All rights reserved.

Automation is important because it brings consistency to what would otherwise be a manual process. Once built, automations can run on demand, on a schedule, or based on a check-in event. While there are many tools you could use to implement the automation, Azure Pipelines and GitHub Actions both have pre-built Power Platform task and action support from Microsoft.

Source control is a feature of Azure DevOps and GitHub that allows you to keep track of versions of your solution components. As you change a Power App, for example, and the solution is exported and checked into source control, you created a new version. If you use the work items to track the change that you made it could be associated with the version when it is checked into source control. This allows a project team to track when changes were made and if necessary, roll back changes that were problematic. Source control can also provide ways for a current production version to be serviced while at the same time a development team is working on a newer version.

Source control is an important part of ALM because it becomes the source of truth for your solution. This means that if your development environment was destroyed, it wouldn't be a problem because you would go to source control to rebuild a new development environment. Source control makes development environments disposable. All deployments to test and production environments originate from the source-controlled version and not an untracked development environment.

While technically solution files exported from an environment can be checked in to source control, it is not an effective way to track your individual component changes. Solution files are a single compressed binary file. If you put them in source control, they would just be able to tell you that the file changed and not allow you to easily enumerate what is changed within each component. To improve this, a step is added to the automation that takes the solution from the development environment and puts it into source control and in the process unpacks the compressed file. By unpacking the compressed solution file, individual files are created for each component (in some cases multiple files for each component) and are checked into source control allowing much more granular tracking of changes. Using these individual files for the components also opens up the possibility of having multiple development environments that are tracking changes in the source control and minimizing cases

of one change conflicting with another.

What is source control?

- 1** A source control system allows developers to collaborate on code and track changes. Use version control to save your work and coordinate code changes across your team.
- 2** The version control system saves a snapshot of your files (history) so that you can review and even roll back to any version of your code with ease.
- 3** Source control protects source code from catastrophe and the casual degradation of human error and unintended consequences.

© Copyright Microsoft Corporation. All rights reserved.

Implement source control for code components

```
1 ## Ignore Visual Studio temporary files, build results, and
2 ## files generated by popular Visual Studio add-ons.
3 ##
4 ## Get latest from https://github.com/github/gitignore/blob/master/VisualStudio.gitignore
5
6 # User-specific files
7 *.ruuser
8 *.suo
9 *.user
10 *.useroscache
11 *.xln.docstates
12
13 # User-specific files (MonoDevelop/Xamarin Studio)
14 *.userprefs
15
16 # Mono auto generated files
17 mono_crash.*
18
19 # Build results
20 [Dd]ebug/
21 [Dd]ebugPublic/
22 [Rr]elease/
23 [Rr]eleases/
24 x64/
25 x86/
26 [Ww][Ii][Nn]32/
27 [Aa][Rr][Mm]/
28 [Aa][Rr][Mm]64/
29 bin/
30 [Bb]in/
31 [Oo]bj/
32 [Oo]jet/
33 [Ll]og/
34 [Ll]ogs/
35
36 # Visual Studio 2015/2017 cache/options directory
37 *.vsl/
38 # Uncomment if you have tasks that create the project's static files in wwwroot
39 #wwwroot/
40
```

© Copyright Microsoft Corporation. All rights reserved.

- In-scope component types:
 - Plug-ins
 - Web Resources (JavaScript)
 - TypeScript (client scripting / PCF controls)
- Should be stored in source control alongside any other component types (e.g. solution files)
- Check .gitignore settings
- Additional considerations:
 - Centralize build operations:
 - Azure Pipelines
 - GitHub Actions
 - Static code analyzers
 - Roslyn
 - GitHub Advanced Security
 - OWASP Source Code Analysis Tools

All custom code components described in this course so far should be factored in as part of a source control strategy, to realise the benefits discussed in the previous slide:

- **Plug-ins:** Store the Visual Studio solution file with all required dependencies. If using SNK files for assembly signing, it's recommended not to password enable these unless strictly necessary, as this can cause issues with other developers
- **Web Resources:** JavaScript are the most important types to store, but CSS, image and Resx files could be stored as well. Care should be taken with larger file sizes.
- **TypeScript:** Store the entire PCF control project file generated via the PAC CLI. Review and check the .gitignore settings to avoid solution bloat

What is .gitignore?

Git shouldn't track every file in your project. Temporary files from your development environment, test outputs, and logs are all examples of files that probably don't need to be tracked.

You can tell Git not to track certain files in your project by adding and configuring a .gitignore file. Entries in a .gitignore file apply only to untracked files. They don't prevent Git from reporting changes to tracked files. Tracked files are files that were committed and exist in the last Git snapshot.

If setting up the repo from Azure DevOps, you can initialize a default .gitignore file by selecting the **VisualStudio** option. This will be recommended for most scenarios but may need additional tweaking for PCF control projects.

Additional considerations

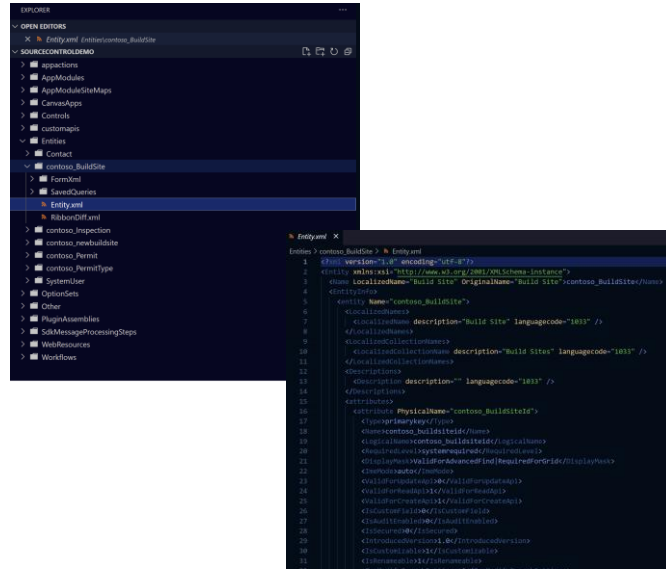
- **Centralize build operations:** By validating that code / solutions build on central servers, you rule out any potential dependency issues or factors on a developers local machine, thereby ensuring that the code can always be built successfully by any developer. On code-check in, you should enforce some type of validation

build.

- **Static code analyzers:** Analyzers allow you to detect potential style, quality and security issues with custom code. Similar to the previous point, there should be a validation step as part of a centralized build that runs a static code analyzer to flag any issues. There are many analyzer tools available; in-depth discussion of these is beyond the scope of the course (more of an AZ-400 topic if the students are interested).

Implement source control for Dataverse solutions

- Setup code repository
 - Recommend Git, using either GitHub or Azure DevOps
- Identify most appropriate tooling to use:
 - SolutionPackager tool
 - Power Platform CLI (recommended)
 - Power Platform Build Tools
- Unpack solution components
 - Can be done locally or automated (e.g. pipeline)
 - Managed, unmanaged or both
- Additional considerations:
 - Branching process
 - GitFlow
 - Collaborating with multiple developers
 - Merging / pull requests



© Copyright Microsoft Corporation. All rights reserved.

The steps to setup source control for a Dataverse solution are as follows:

- **Setup code repository:** In theory, this could be a Git repository from any vendor, but Azure DevOps or GitHub is recommended. Team Foundation Version Control is now considered legacy and it's not recommended to use.
- **Identify most appropriate tooling:** This will depend on preference and ultimately whether automation will be a consideration:
 - **SolutionPackager tool:** Useful for manual unpacking of solution components on a Windows machine. Could also be called via custom scripting. Not cross-platform and no longer recommended.
 - **Power Platform CLI:** Includes all of the capabilities in the SolutionPackager tool, plus additional capabilities and has cross-platform support. Useful for both manual and automated executions
 - **Power Platform Build Tools:** Effectively, the same as Power Platform CLI, but used from Azure DevOps with full support for automation. Can negate the need for writing custom scripts.
- **Unpack solution components:** By running the appropriate command (e.g. pac solution unpack), the contents of the zip file will be extracted into a logical structure. The expanded contents will include a mixture of XML, JSON and other file types, as indicated in the screenshots. It is always desirable to store the unmanaged contents in the repo but may also be necessary to store the managed contents as well (e.g. there is no shared development environment that can be used to generate managed solutions).

INSTRUCTOR NOTE: This could be a good opportunity to challenge the students by asking why unpacking is so desirable, as opposed to just storing the .zip file in Git. Main answer is the ability to see how discrete components have changed over time.

Additional considerations (impossible to go into depth here) regarding source control:

- What is the branching process for the project? Branches should always be temporary in nature and disposed of as soon as practically possible. Considering implementing a formal process, such as Git flow:
 - <https://nvie.com/posts/a-successful-git-branching-model/>

- If multiple developers are involved, how does the process look? Separate developer environments and merging of changes on an agreed frequency? What automation is required here?
- How is solution quality ensured? Pull requests can provide an excellent mechanism in enforcing regular peer reviews.
- If there is a conflict, how are changes merged in? This can be particularly challenging when working with XML / JSON files – what process should you follow to validate you haven't "broke the build" and made the solution undeployable?

Automating ALM with Microsoft Power Platform CLI

- Set of command line actions that can be used to automate all aspects of a deployment
- Install via:
 - Direct download - <https://aka.ms/pac>
 - [Power Platform Tools VS Code Extension](#)

```

PS C:\Users\> pac
Microsoft PowerPlatform CLI
Version: 1.10.0-Preview
Online documentation: https://aka.ms/PowerPlatformCLI
Feedback, suggestions, issues: https://github.com/microsoft/powerplatform-build-tools/discussions

Usage: pac [admin] [application] [auth] [canvas] [catalog] [connection] [connector] [copilot] [data] [env] [help] [modelbuilder] [package] [pages] [pcf] [pipeline] [plugin] [power-fx] [solution] [telemetry]
[ry] [test] [tool]

admin                Work with your Power Platform Admin Account
application           Commands for listing and installing available Dataverse applications from AppSource
auth                 Manage how you authenticate to various services
canvas               Operating with Power Apps .csapp files
catalog              Commands for working with Catalog in Power Platform
connection            Commands for working with Dataverse connection
connector             Commands for working with Power Platform Connectors
copilot              Tools and utilities for copilot management
data                 Import and export data from Dataverse
env                  Work with your Dataverse organization
help                 Show help for the Microsoft Power Platform CLI
modelbuilder          Code generator for Dataverse data and tables
package              Commands for working with Dataverse package projects
pages                Commands for working with Power Pages website
pcf                  Commands for working with Power Apps component framework projects
pipeline             Work with Pipelines
plugin               Commands for working with Dataverse plug-in class library
power-fx             (Preview) Commands for working with Power Fx
solution              Commands for working with Dataverse solution projects
telemetry             Manage telemetry settings
test                 (Preview) Execution of automated tests for a Power App
tool                 Power Platform tools that can be installed and launched.
  
```

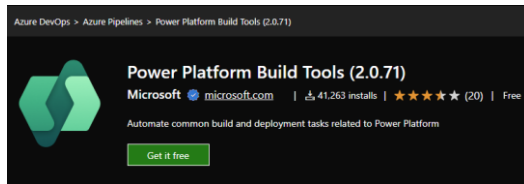
© Copyright Microsoft Corporation. All rights reserved.

Microsoft Power Platform CLI is a simple, one-stop developer CLI that empowers developers and ISVs to perform various operations in Microsoft Power Platform related to environment lifecycle, authentication, Microsoft Dataverse environments, solution packages, Power Pages, code components. Specifically, for ALM, the following commands are particularly relevant:

- `pac admin` – Allows for environment backups, copy, restore, resets and more. Useful for prepping an environment before a deployment
- `pac application` – Lists and allows you to install Power Platform / Dynamics 365 applications into an environment. Useful for ensuring required dependent components are present in a new environment before deploying any custom solutions
- `pac auth` – Allows you to authenticate into a specific Dataverse environment or tenant
- `pac data` – Allows you to export / import data, based on a CMT schema definition.
- `pac env` – Allows you to return a list of environments and perform discrete actions against them, such as updating environment settings.
- `pac package` – Provides actions relating to package deployments, including creating and deploying to a environment
- `pac pipeline` – For extensibility scenarios involving Power Platform pipelines. Allows you to list and trigger a pipeline deployment
- `pac solution` – Various actions relating to solutions, including creation, unpacking / packing, export/import and more.
- `pac tool` – Provides access to the latest versions of official tools, such as the PRT, CMT and package deployer tool.

The PAC CLI is updated regularly and is used as the backbone of the Power Platform Build Tools (next side). It's therefore of imperative importance that developers understand and work with it regularly.

Automating ALM with Power Platform Build tools



- Set of tasks that can be used in an Azure DevOps build or release pipeline to:
 - Deploy solutions
 - Manage environments
 - Import / export data
- Leverages PAC CLI capabilities
- Advantages:
 - Supports more complex scenarios
 - Can integrate with Azure Repos and Azure Boards
- Disadvantages:
 - Requires development effort / expertise (YAML)

© Copyright Microsoft Corporation. All rights reserved.

Use Microsoft Power Platform Build Tools to automate common build and deployment tasks related to apps built on Microsoft Power Platform. These tasks include:

- Synchronizing solution metadata (also known as solutions) that contains the various platform components such as:
- Customer engagement apps: Dynamics 365 Sales, Customer Service, Field Service, Customer Insights - Journeys, and Project Operations
- Canvas apps
- Model-driven apps
- UI flows
- Virtual agents
- AI Builder models
- Connectors between development environments and source control
- Generating build artifacts
- Deploying to downstream environments
- Provisioning or deprovisioning environments
- Performing static analysis checks against solutions by using the Power Apps checker service

Use Microsoft Power Platform Build Tools tasks with any other available Azure DevOps tasks to compose your build and release pipelines. Pipelines that teams commonly put in place include Initiate, Export from Dev, Build, and Release.

Summary



- Solutions are the basis for ALM in Power Platform
- ALM should be planned and implemented at the start of each project
- A successful deployment must include planning and collaboration between the developers and the operations team

© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Solutions and Application Lifecycle Management



© Copyright Microsoft Corporation. All rights reserved.